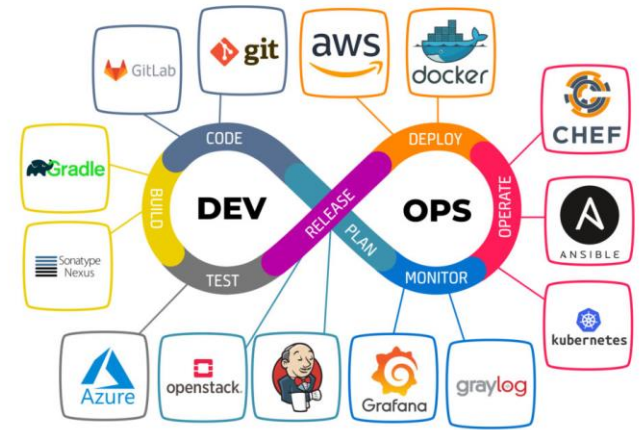


Gestion des versions décentralisées



CHAPITRE 1

GIT : SYSTÈME DE CONTRÔLE DES VERSIONS



Objectifs

- ✓ Comprendre les principes fondamentaux des VCS et leur importance dans le développement de logiciels.
- ✓ Savoir utiliser Git pour suivre les modifications de code et travailler avec des branches et des révisions.
- ✓ Comprendre comment utiliser Git pour collaborer efficacement avec d'autres développeurs sur un projet.
- ✓ Être capable de résoudre les conflits de fusion et d'éviter les pertes de données.
- ✓ Comprendre comment utiliser GitHub pour héberger des dépôts Git et faciliter la collaboration.

Qu'est-ce que Git ?



- ❖ Git est **un système de gestion de version décentralisé open-source**, créé en 2005 par Linus Torvalds, le créateur de Linux.
- ❖ Il permet de **suivre les modifications** apportées à un ensemble de **fichiers au fil du temps**.
- ❖ Les développeurs peuvent travailler sur leur **propre branche** de développement et **fusionner** les **modifications apportées par les autres**, ce qui facilite la collaboration.
- ❖ Git permet de **résoudre les conflits** qui peuvent survenir lors de la **fusion de modifications**.
- ❖ Git est rapide, fiable et peut être utilisé pour **gérer des projets** de toute taille.
- ❖ Git est compatible avec une large gamme d'outils et de services de développement, ce qui en fait un choix populaire pour la gestion de version de code.

Systeme de Contrôle de Version SCV (1)

- **Suivi des modifications** : Les SCVs enregistrent chaque version et les modifications apportées, ce qui permet aux développeurs de retrouver facilement des versions antérieures du code et de comprendre son évolution.
- **Collaboration** : Les SCVs permettent à chaque développeur de travailler sur sa propre branche, ce qui facilite la collaboration et la fusion des modifications apportées.
- **Gestion des conflits** : Les SCVs permettent de gérer les conflits qui peuvent survenir lors de la fusion de modifications apportées par différents développeurs.
- **Sauvegarde** : Les SCVs offrent une sauvegarde fiable et efficace de l'ensemble du code source et de son historique, ce qui réduit les risques de perte de données.
- **Développement itératif** : Les SCVs permettent de travailler de manière itérative en travaillant sur des fonctionnalités spécifiques de manière isolée, ce qui facilite le développement par étapes et réduit les risques de régressions.

Système de Contrôle de Version SCV (2)

❖ Types de SCV

Système de Contrôle de Version Centralisé (SCVC)
ou Centralized Version Control System (CVCS)



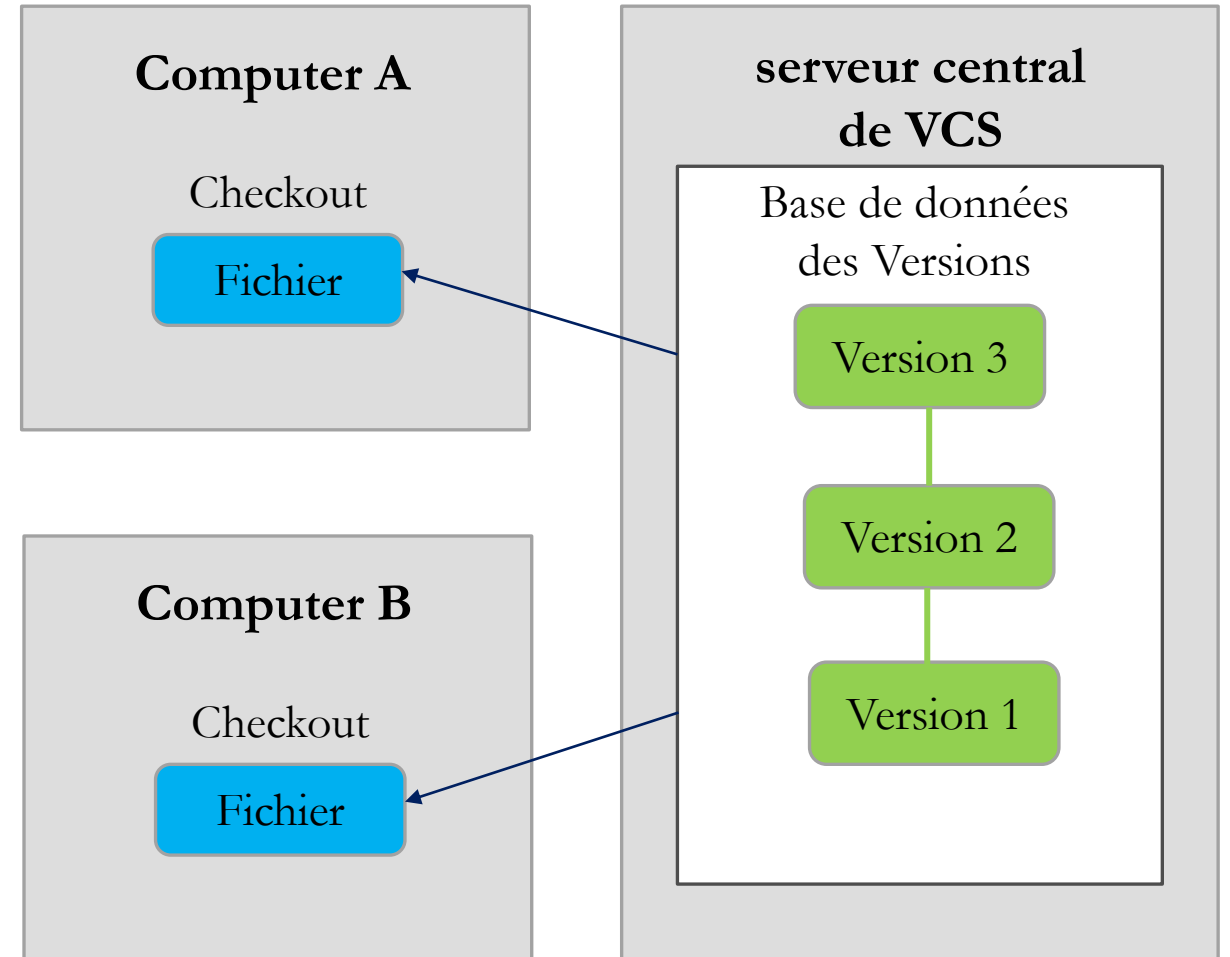
Système de Contrôle de Version Distribué (SCVD)
ou Distributed Version Control System (DVCS)



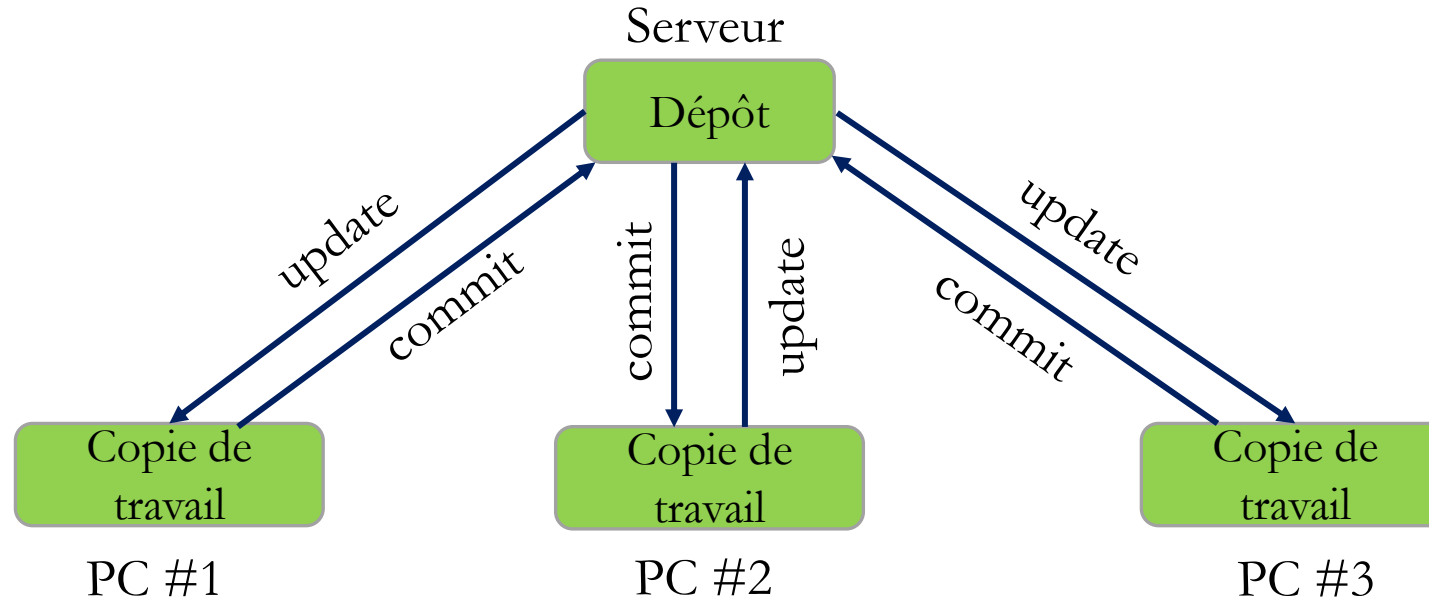
SCV centralisé-SCVC (1)

SCV centralisé

- utilise un serveur central pour stocker tous les fichiers et permet la collaboration d'équipe.
- fonctionne sur un dépôt unique auquel les utilisateurs peuvent accéder directement à un serveur central.



SCV centralisé-SCVC (2)



Ici, chaque poste de travail est connecté au dépôt central de codes.

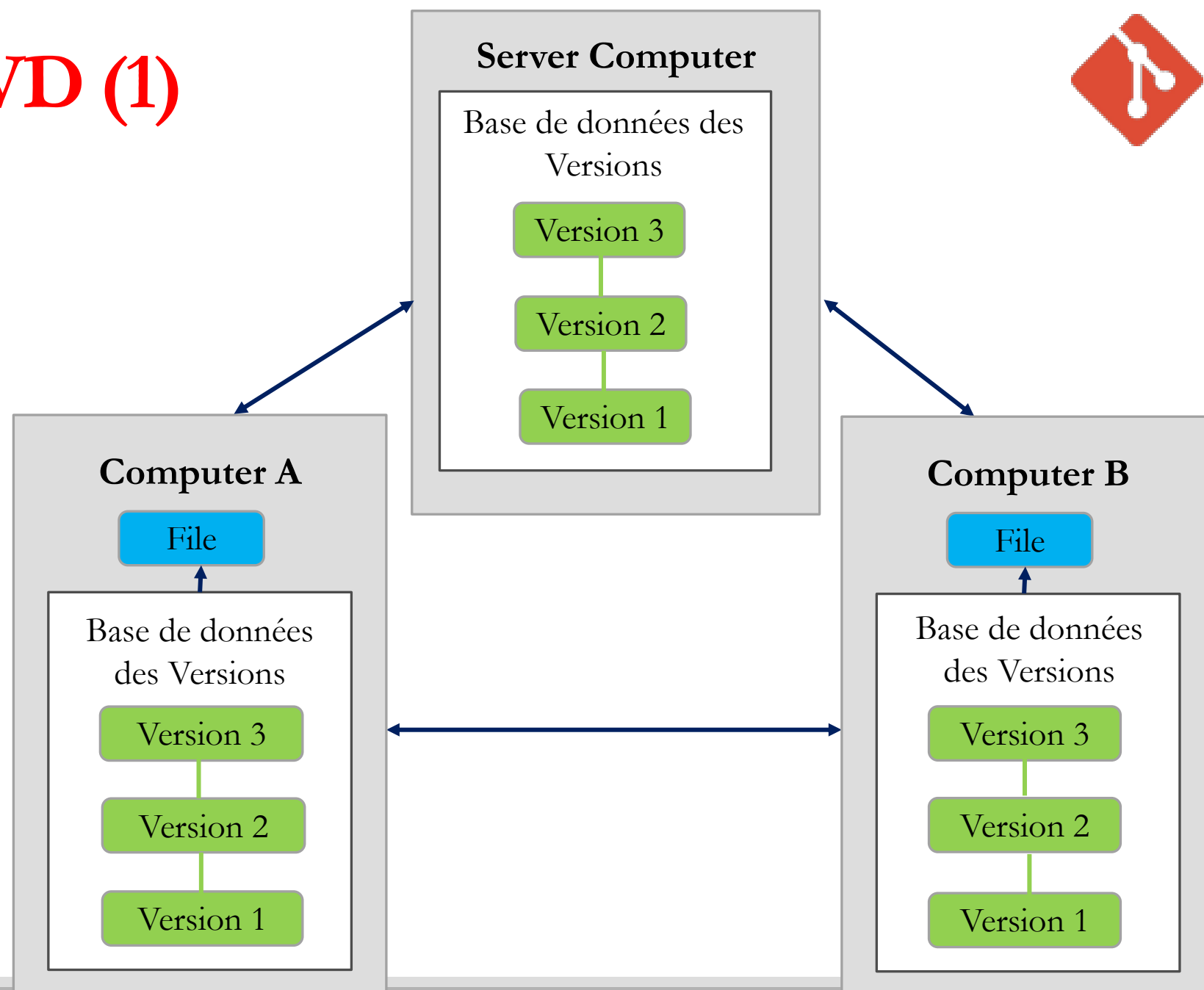
Inconvénients - Il n'est pas disponible localement. 😞

Le crash de Système de Contrôle de Version Centralisé entraînera la perte de toutes les données du projet. 😞

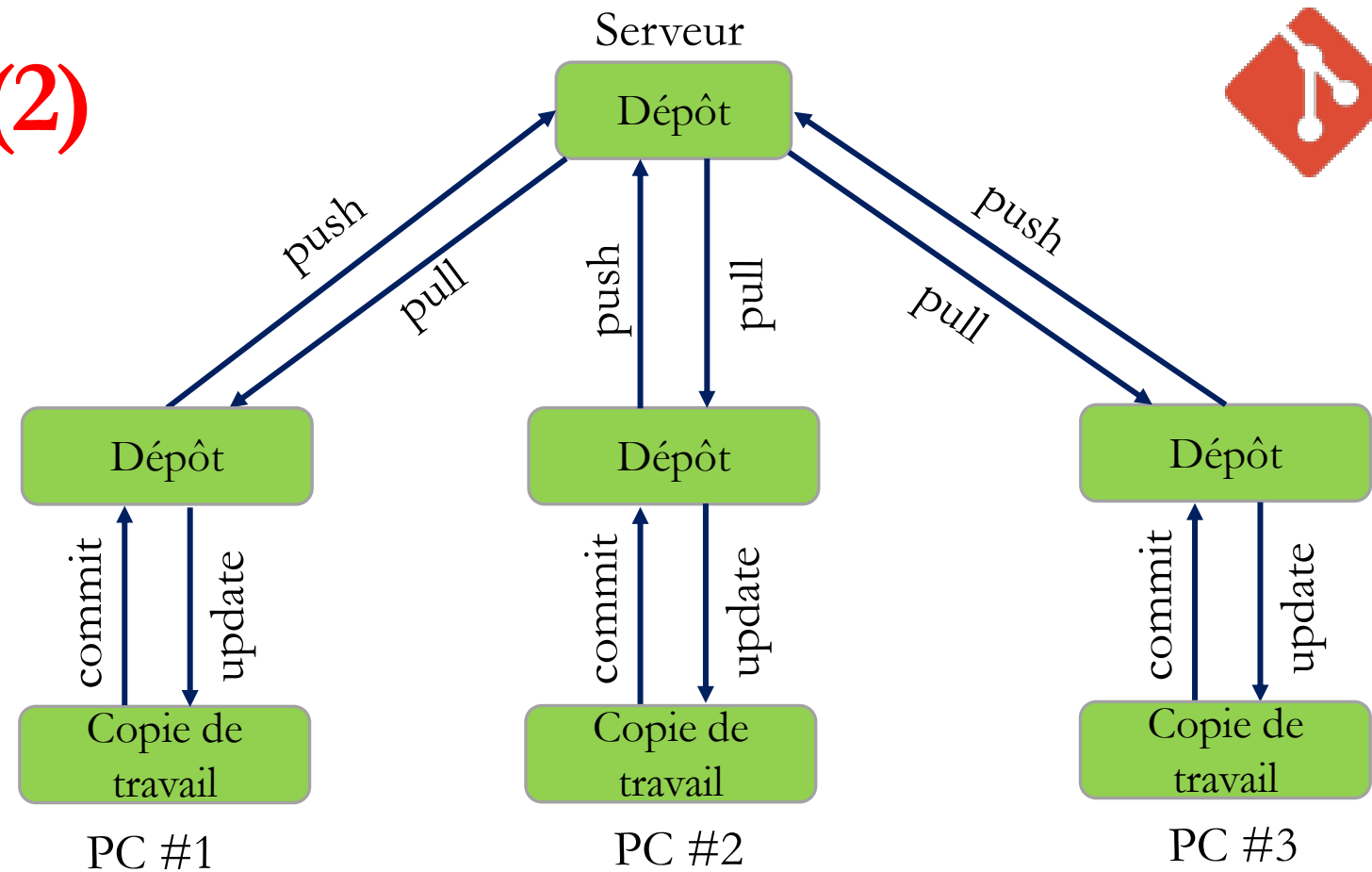
SCV distribué-SCVD (1)



- ❖ VCS distribué ou Git
- ❖ Dans le système de contrôle de version distribué, chaque contributeur dispose d'une copie locale ou « clone » du dépôt principal.
- ❖ L'utilisateur peut modifier et valider le dépôt local sans aucune **interférence**.
- ❖ L'utilisateur peut mettre à jour son dépôt local à partir du serveur central.
- ❖ L'utilisateur peut mettre à jour le serveur central à partir de son dépôt.



SCV distribué-SCVD (2)



Les opérations dans SCVD sont rapides.

De nouvelles **modifications** peuvent être effectuées **localement** sans manipuler les données **centrales**.

Si le serveur central tombe en panne à tout moment, les données perdues peuvent être **facilement récupérées** à partir de l'un des dépôts locaux du contributeur.

Qu'est-ce que GitHub?

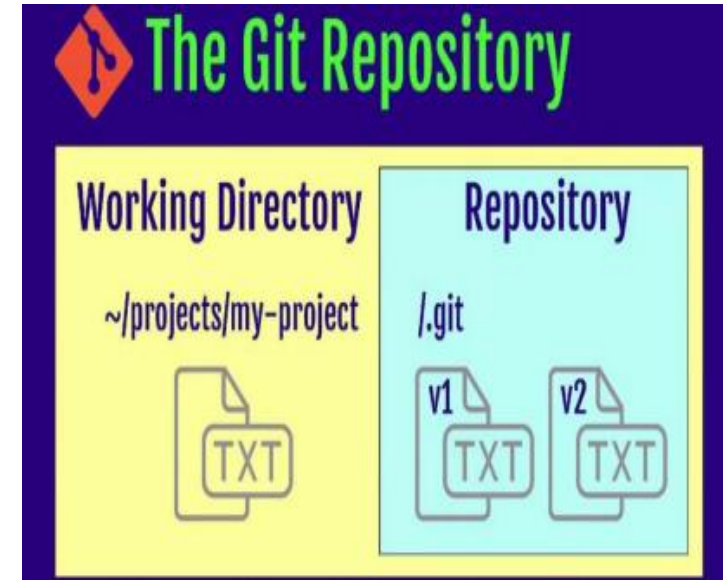


- ❖ GitHub est une plateforme en ligne basée sur Git.
 - ❖ Il permet la gestion de projet collaborative.
 - ❖ GitHub offre un hébergement gratuit pour les dépôts Git.
 - ❖ Il facilite la collaboration entre les développeurs.
 - ❖ Il fournit des outils de communication tels que les commentaires, les notifications et les discussions.
 - ❖ GitHub est largement utilisé pour l'hébergement de projets open source.
- ➔ Pour récapituler, Git est un logiciel de gestion de version tandis que GitHub est un service en ligne d'hébergement de dépôts Git qui fait office de serveur central pour ces dépôts.

Fonctionnement de base de Git (1)

Démarrer un Dépôt Git

- Un “dépôt” correspond à la copie et à l’importation de l’ensemble des fichiers d’un projet dans Git.
- Il existe deux façons de créer un dépôt Git :
 - On peut importer un répertoire déjà existant dans Git,
 - On peut cloner un dépôt Git déjà existant.



Fonctionnement de base de Git (2)

La gestion des informations selon Git

- Git examine les données à la manière d'un flux d'instantanés ou “snapshots”.
- A chaque fois qu'on va **valider ou enregistrer** l'état d'un projet dans Git, il va prendre un instantané du contenu de l'espace de travail à ce moment et **va enregistrer une référence** à cet instantané pour qu'on puisse y accéder par la suite.
- Chaque **instantané est stocké** dans une base de **données locale**, c'est-à-dire une base de données **située sur notre propre machine**.
- Le fait que l'on dispose de l'historique complet d'un projet localement fait que la grande majorité des opérations de Git peuvent être réalisées localement, c'est-à-dire **sans avoir à être connecté à un serveur central distant**.
- Cela rend les opérations beaucoup plus rapides et le travail de manière générale beaucoup plus agréable.

Fonctionnement de base de Git (3)

Les états des fichiers

- Comment Git fait-il pour suivre les modifications sur les fichiers d'un projet ?
- ➔ Pour comprendre cela, il faut savoir qu'un fichier peut avoir deux grands états dans Git :
 - il peut être **sous suivi** de version ou **non suivi**.
- Un fichier possède l'état "**suivi**" si il appartenait au dernier instantané capturé par Git, c'est-à-dire si il est enregistré en base.
- Tout fichier qui n'appartenait pas au dernier instantané et qui n'a pas été indexé est "**non suivi**".

Fonctionnement de base de Git (4)

Les états des fichiers

- Lorsqu'on démarre un dépôt Git en important un répertoire déjà existant depuis notre machine, les fichiers sont au départ tous non suivis. On va donc déjà devoir demander à Git de les indexer et de les valider (les enregistrer en base).
- Lorsqu'on clone un dépôt Git déjà existant, c'est différent puisqu'on copie tout l'historique du projet et donc les fichiers sont tous déjà suivis par défaut.
- Ensuite, chaque fichier suivi peut avoir l'un de ces trois états :
 - Modifié (“modified”) ;
 - Indexé (“staged”) ;
 - Validé (“committed”).

Fonctionnement de base de Git (5)

Les états des fichiers

- Lors du démarrage d'un dépôt Git à partir d'un **dépôt local**, on demande à Git **de valider** l'ensemble des fichiers du projet.
 - ➔ Un fichier est “**validé**” lorsqu'il est **stocké dans la base de données locale**.
 - ➔ Lors du **clonage** d'un dépôt déjà **existant**, les fichiers **sont enregistrés par défaut** en base et ils sont donc **validés par défaut**.
- Ensuite, lorsqu'on va travailler sur notre projet, on va certainement ajouter de nouveaux fichiers ou modifier des fichiers existants.
 - ➔ Les fichiers **modifiés** vont être considérés **comme “modifiés”** par Git tandis que les nouveaux fichiers vont être “non suivis”.
 - ➔ **Un fichier modifié est considéré comme “modifié” par Git tant qu'il n'a pas été indexé.**

Fonctionnement de base de Git (6)

Les zones de travail

- Les états de fichiers sont liés à des zones de travail dans Git.
- En fonction de son état, un fichier va pouvoir apparaître dans telle ou telle zone de travail.
- Tout projet Git est composé de trois sections :
 - le répertoire de travail (working tree),
 - la zone d'index (staging area),
 - le répertoire Git (repository).

Fonctionnement de base de Git (7)

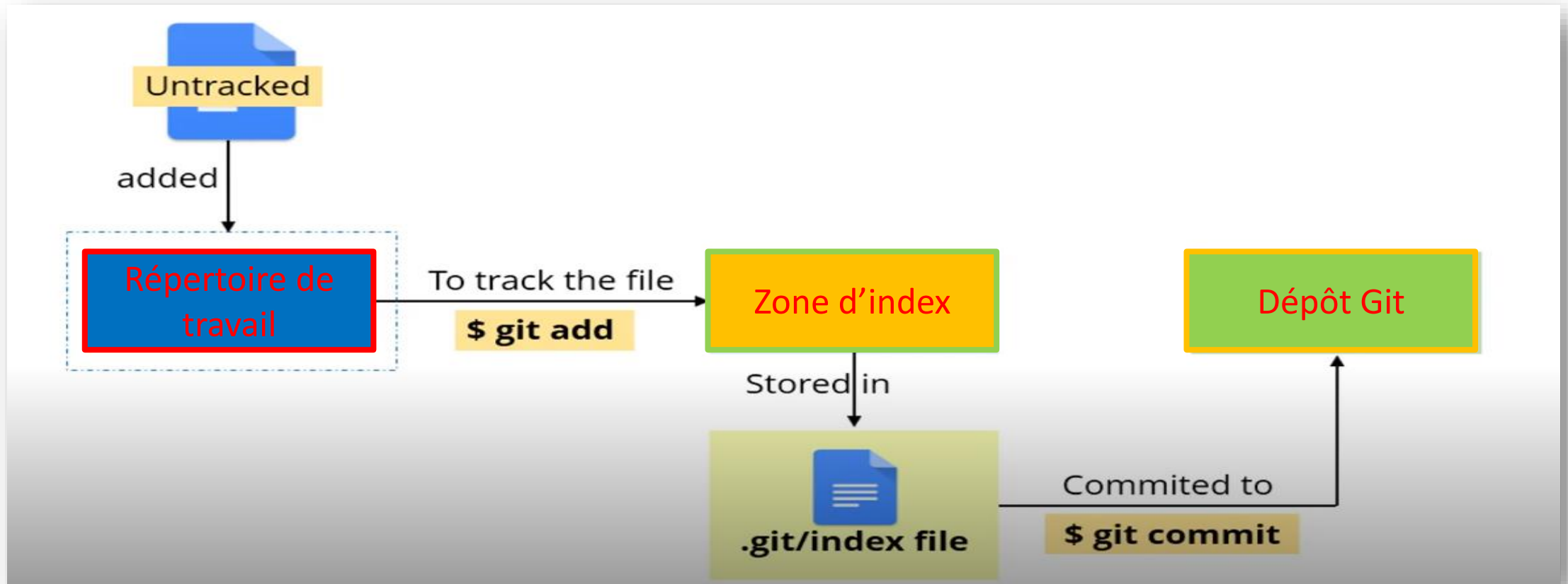
Les zones de travail

- ❖ **Le répertoire de travail ou “working directory”** : correspond à une extraction unique (“checkout”) d’une version du projet. Les fichiers sont extraits de la base de données compressée située dans le répertoire Git et sont placés sur le disque afin qu’on puisse les utiliser ou les modifier.
- ❖ **La zone d’index ou “staging area”** : correspond à un simple fichier, généralement situé dans le répertoire Git, qui stocke les informations concernant ce qui fera partie du prochain instantané ou du prochain “commit”.

zone est un intermédiaire entre le répertoire de travail et le dépôt.
- ❖ **Le répertoire Git “Repository”** : est l’endroit où Git stocke les méta-données et la base de données des objets de votre projet. C’est la partie principale ou le cœur de Git.

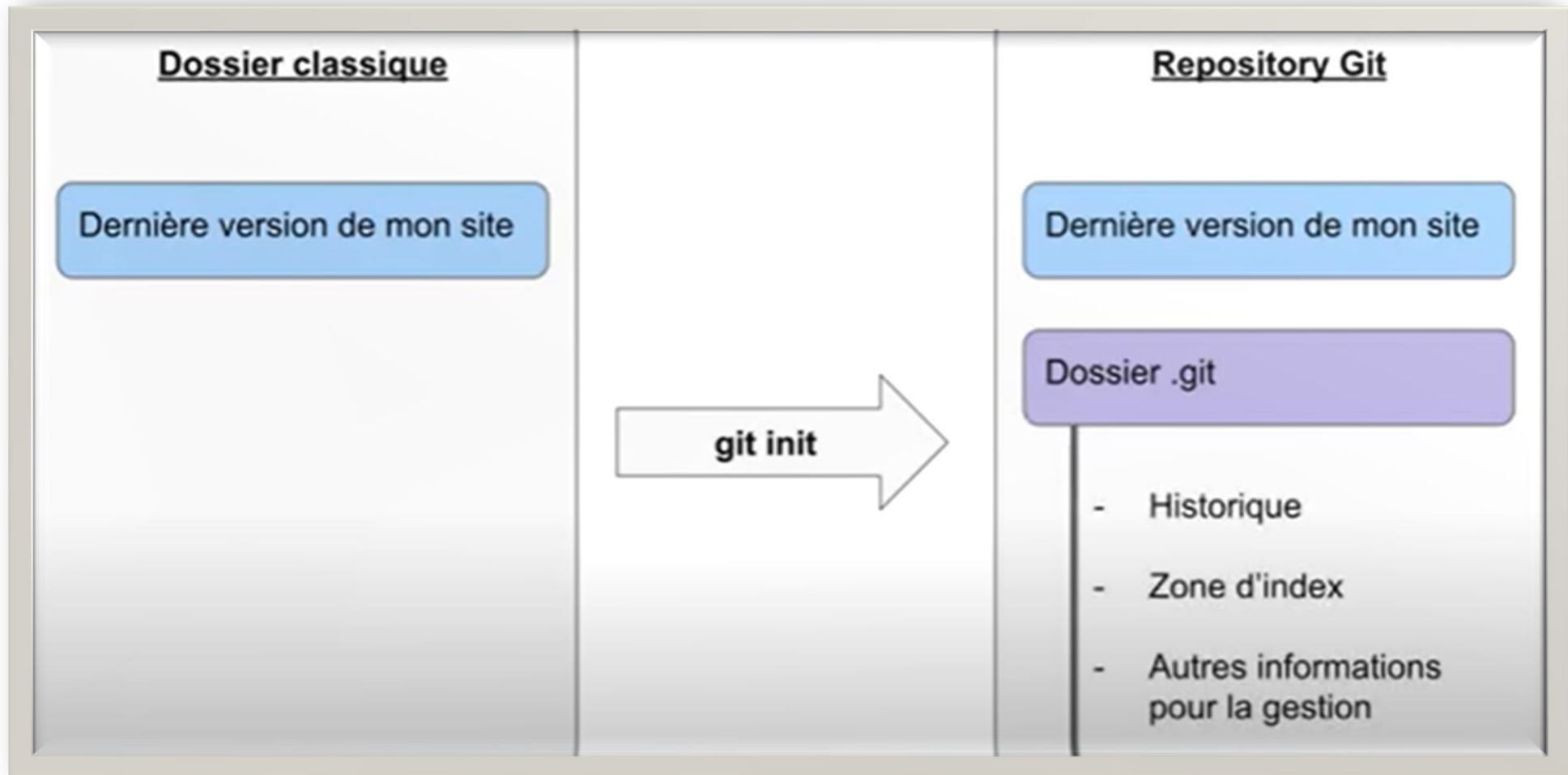
Fonctionnement de base de Git (8)

Les zones de travail



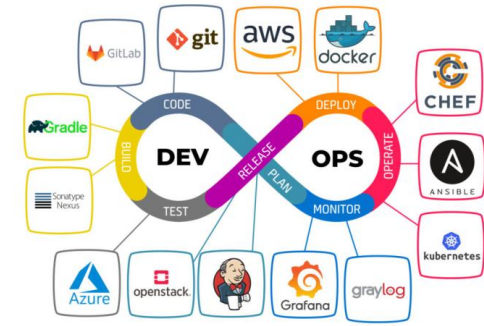
Fonctionnement de base de Git (9)

Les zones de travail



Notions (1)

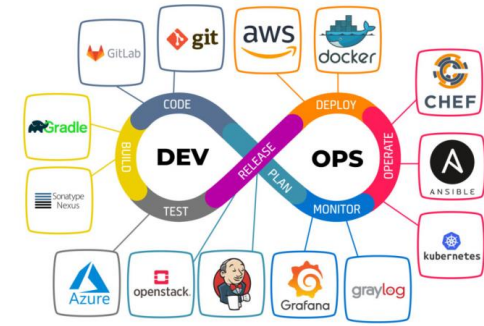
- ❖ Dépôt (repository)
- ❖ Commit
- ❖ Copie de travail
- ❖ Index



Notions (2)

Dépôt (repository)

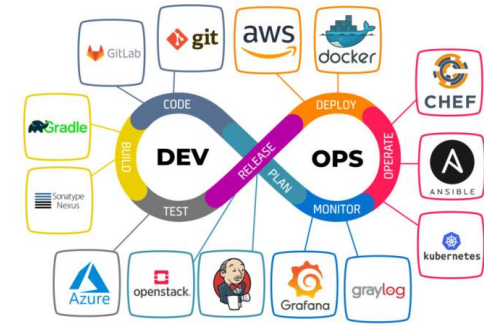
- ❖ Le répertoire caché `.git` est nommé dépôt (repository).
- ❖ Il contient **toutes les données** dont GIT a besoin pour **gérer l'historique**.
- ❖ Sauf par exceptions, on **ne peut pas modifier** son contenu directement, mais uniquement en passant par **les commandes GIT**.



Notions (3)

Commit

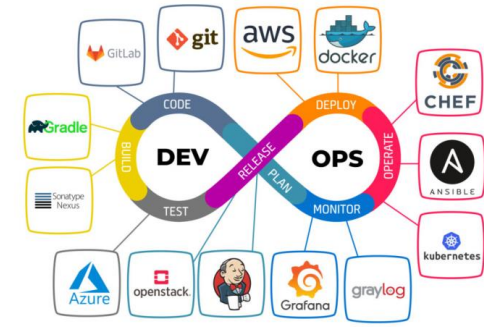
- ❖ L'historique d'un projet est **une séquence de « photos »**, contenant l'état de tous les fichiers du projet.
 - ❖ Ces « photos » s'appellent des commits, et possèdent :
 - une date
 - un auteur
 - une description textuelle
 - un lien vers le(s) commit(s) précédent(s)
- ➔ On parle également de révision.



Notions (4)

Remarque importante

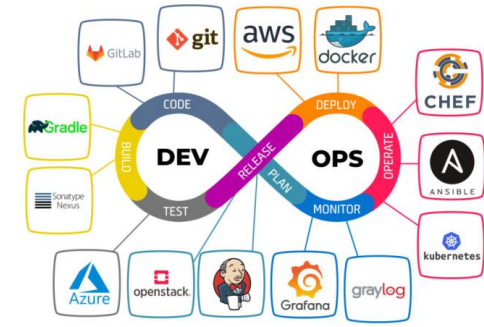
- ❖ Bien que conceptuellement, chaque commit contient tous les fichiers du projet, GIT utilise un système de stockage très efficace :
 - le commit ne stocke que les fichiers modifiés par rapport au commit précédent;
 - l'ensemble peut être compressé (`git gc`) pour réduire encore la redondance (lorsque deux versions successives d'un fichier sont très proches).



Notions (5)

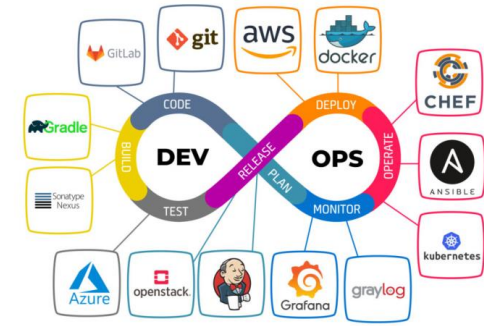
Copie de travail

- ❖ On appelle copie de travail (working copy) les fichiers effectivement présents dans le répertoire géré par GIT.
- ❖ Leur état peut être différent du dernier commit de l'historique.



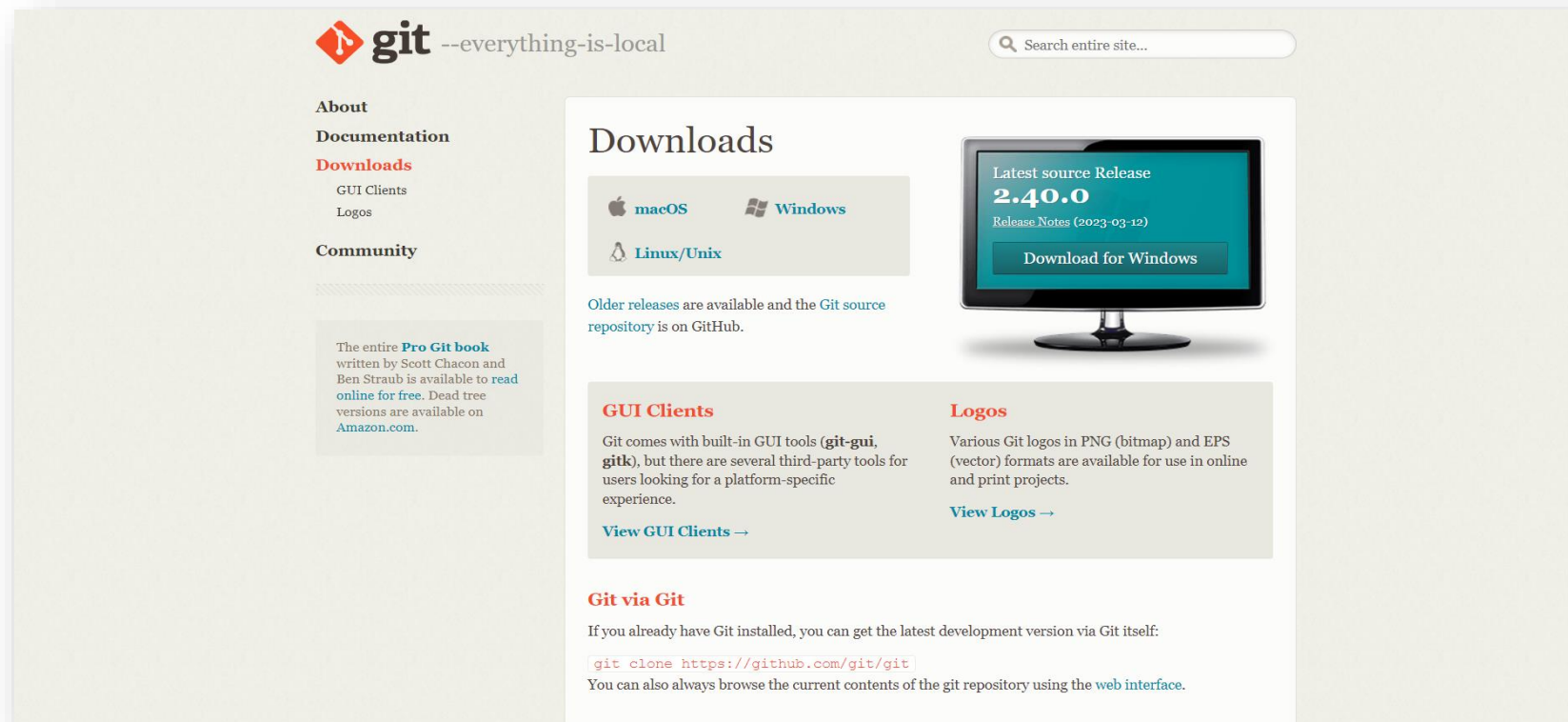
Notions (6)

L'Index



- ❖ L'index est un espace temporaire contenant les modifications prêtes à être « committées ».
- ❖ Ces modifications peuvent être :
 - création de fichier
 - modification de fichier
 - suppression de fichier

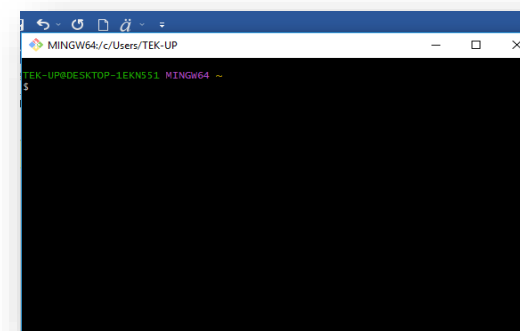
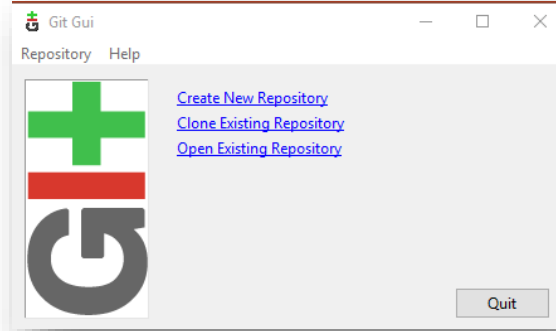
Installation de Git (1)



Sous Windows, télécharger la dernière version de Git : <https://gitforwindows.org/>.

Installation de Git (2)

Vérifier l'installation



Git GUI : l'interface graphique d'un outil en ligne de commande.

Git bash : permet aux utilisateurs Windows de travailler avec les fonctions de Git **en lignes de commandes**.

Git CMD : est un ensemble d'utilitaires de ligne de commande conçus pour s'exécuter dans un environnement de ligne de commande de type Unix.

Installation de Git (3)

Vérifier l'installation

Lancer le terminal “git bash”.

Vérifions que l'installation s'est bien déroulée.

`git version` → permet d'afficher le numéro de version de Git.

Si l'installation c'est bien déroulée :

```
user@DESKTOP-RRVCC4I MINGW64 ~  
$ git version  
git version 2.39.2.windows.1
```

Paramétrage de Git (1)

Configuration de l'environnement

- ❖ Personnaliser votre environnement Git.
- ❖ Vous ne devriez avoir à réaliser ces réglages qu'une seule fois.
- ❖ Vous pouvez aussi les changer à tout instant en relançant les mêmes commandes.
- ❖ Pour la configuration, nous allons utiliser :
git config → permet de voir et modifier les variables de configuration qui contrôlent tous les aspects de l'apparence et du comportement de Git.

```
$ git config --global user.name "Maha Cherif"  
$ git config --global user.email "maha.cherif@gmail.com"
```

Paramétrage de Git (2)

- ❖ Si vous souhaitez vérifier vos réglages, vous pouvez utiliser la commande `git config --list` pour lister tous les réglages que Git a pu trouver jusqu'ici :

```
$ git config --list
user.name=Maha Cherif
user.email=maha.cherif@gmail.com
color.status=auto
color.branch=auto
color.interactive=auto
color.diff=auto
...
```

- ❖ Vous pouvez aussi vérifier la valeur effective d'un paramètre particulier en tapant `git config <paramètre>` :

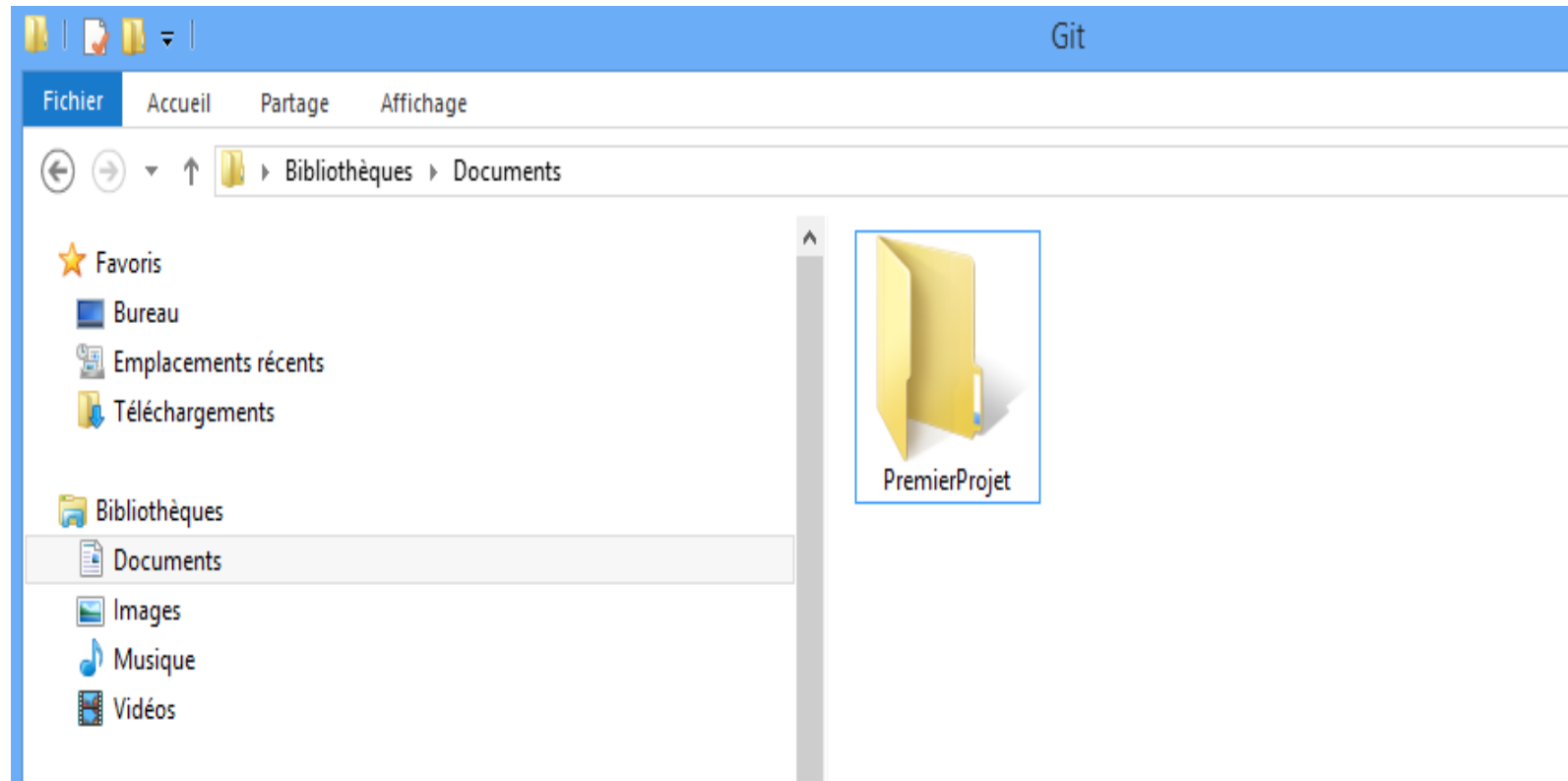
```
$ git config user.name
Maha Cherif
```

Démarrer un dépôt Git: Créer un nouveau dépôt vide

Dans un premier temps, créez un dossier sur votre ordinateur (avec le nom de notre projet, c'est mieux).

Je vous conseille de suivre uniquement les étapes suivantes :

- ➔ Allez dans “Document”.
- ➔ Créez le dossier “PremierProjet”.
- ➔ Accédez à votre dossier.



Initialisation d'un nouveau dépôt : git init (1)

❖ Ensuite, lancez deux lignes de commande dans Git Bash :

`cd documents/PremierProject` % `cd` : sert à se déplacer dans le dossier existant

`git init` % est la seule commande utilisée pour la configuration initiale d'un nouveau dépôt.

❖ L'exécution de cette commande créera un nouveau `.git` sous-répertoire dans notre répertoire de travail actuel.

 MINGW64:/c/Users/user/documents/PremierProject

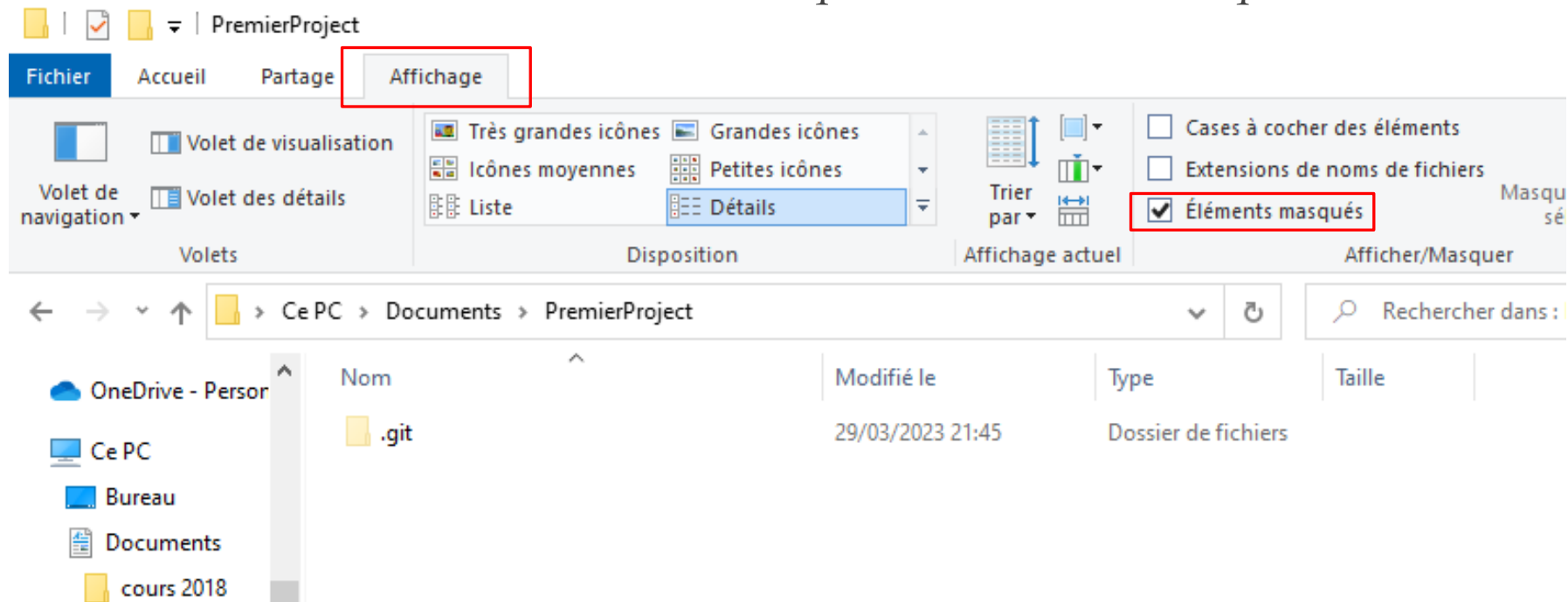
```
user@DESKTOP-RRVCC4I MINGW64 ~
$ cd documents/PremierProject

user@DESKTOP-RRVCC4I MINGW64 ~/documents/PremierProject
$ git init
Initialized empty Git repository in C:/Users/user/Documents/PremierProject/.git/

user@DESKTOP-RRVCC4I MINGW64 ~/documents/PremierProject (master)
$
```

Initialisation d'un nouveau dépôt : git init (2)

- ❖ Vous avez bien initialisé votre dépôt Git.
- ❖ Entrez dans le fichier (dans document)
- ❖ Un dossier caché **.git** a été créé ! Mais, vous pouvez l'afficher en allant dans Affichage
=> et marquez Éléments masqués.



Git : commandes utiles (1)

❖ Nous pouvons également travailler sur **Git CMD**

`mkdir projet` % création d'un projet .git

`cd .\projet` % accéder à l'intérieur de dossier projet

`git init` % initialiser un git et le versionning

`git help config` % ouvre le navigateur « **git-config - Get and set repository or global options** »

`git help name_of_command` % accéder à la commande demandée

`git config --global color.ui true` % améliorer la visibilité des informations par couleur

`git config --list` % afficher la liste des opérations

`touch` % créer fichier .txt dans le dossier

`ls` % pour afficher le contenu du dossier et s'assurer que tout a bien fonctionné

Git : commandes utiles (2)

❖ Créer un fichier dans projet (copie1.txt)

```
$ touch copie1.txt
```

```
git status    % observer qu'il n'est pas suivi
```

➔ Nous remarquons que ce fichier n'est pas tracké « untracked » (c'est que le fichier n'est pas encore ajouté à l'index ou enregistré dans le dépôt local)

```
git add copie1.txt    % ajouter le fichier dans l'index
```

```
git reset copie1.txt    % retirer le fichier de l'index
```

```
git commit -m "mon premier commit" % garder votre code à jour avec les dernières modifications et capturez l'état du projet à ce moment-là
```

```
git commit -a -m "ajout Travailler TP1 dans copie1.txt " % enregistrer la modification
```

➔ Vérifier dans le fichier copie1.txt l'ajout de Travailler TP1

```
cat copie1.txt    % afficher le contenu du fichier copie1.txt
```

Git : commandes utiles (3)

❖ Créer deux fichiers dans projet (copie2.txt et copie3.txt)

git status % observer qu'il n'est pas suivi

➔ Nous remarquons que ce fichier n'est pas tracké

git add *.txt % ajouter les deux fichiers en même temps dans l'index

git commit -m " copie2 et copie3 .txt "

echo "hello first file" >> file1.txt % Créer et modifier le fichier

echo "add new line" >> file2.txt % Créer et modifier le fichier

echo "add another line" > file2.txt % modifier le fichier existant

git add . % ajouter les deux fichiers dans le répertoire

git status % observer qu'il demande de les commiter

git commit -m "my second commit "

Git : commandes utiles (4)

Tracking and Logs

`git commit --amend` % modifier le message ou le fichier de commit déjà existant

`git log` % afficher la liste par défaut des 10 derniers commits

`git log --n` % afficher les n derniers commits

`git log --oneline` % afficher le log de commit

`git log -author= "email or name"` % afficher le commit spécifique du développeur

Git : commandes utiles (5)

❖ D'autres commandes utiles

`git reset <filename>` % Retirer un fichier de l'index

`git diff` % nous permet d'observer les modifications dans la zone de travail et dans l'index

`pwd` % affiche le nom et chemin du répertoire courant. Elle permet de savoir où on se situe sur notre machine

❖ Pour Sortir de la liste et avoir la main, des fois nous sommes obligés de taper `:q` ou `:wq`

...

➔ Réalisation du TP1

Bon travail



- ❖ GitHub est un site Web, un service basé sur le cloud (en ligne) qui aide les développeurs à **stocker** (**héberger**) et à **gérer leur code**, ainsi qu'à suivre et contrôler les modifications apportées à leur code.
- ❖ GitHub est une interface, son rôle est de **faciliter la collaboration** entre les développeurs.
- ❖ GitHub est un **dépôt distant**.

Objectifs

➔ Comment **héberger** mon dépôt local, **accéder à un dépôt distant** et le copier en **local** ?

Créer un dépôt sur un serveur



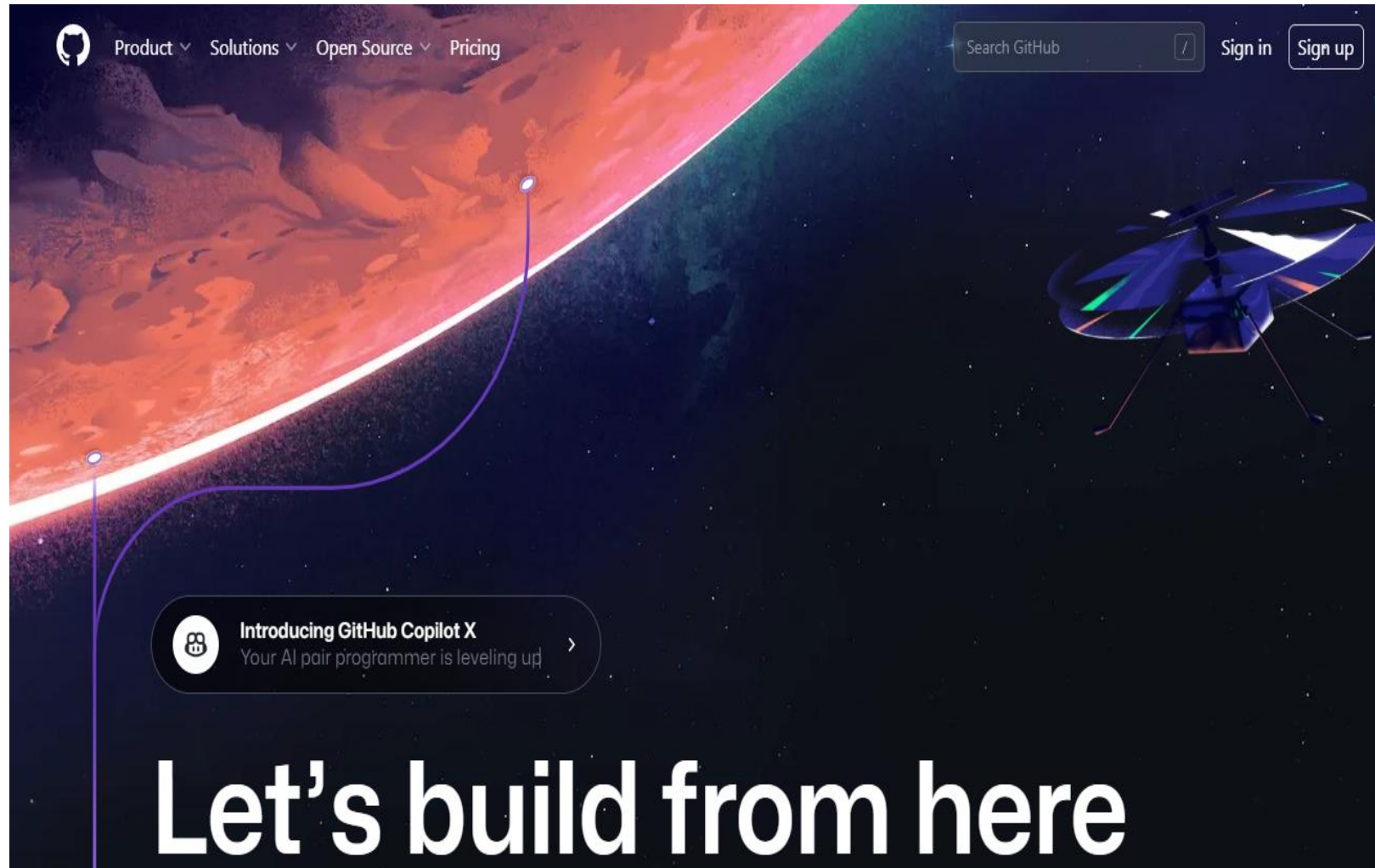
- ❖ Il existe plusieurs sites permettant d'héberger et de partager nos projets Git.





Créer compte GitHub (1)

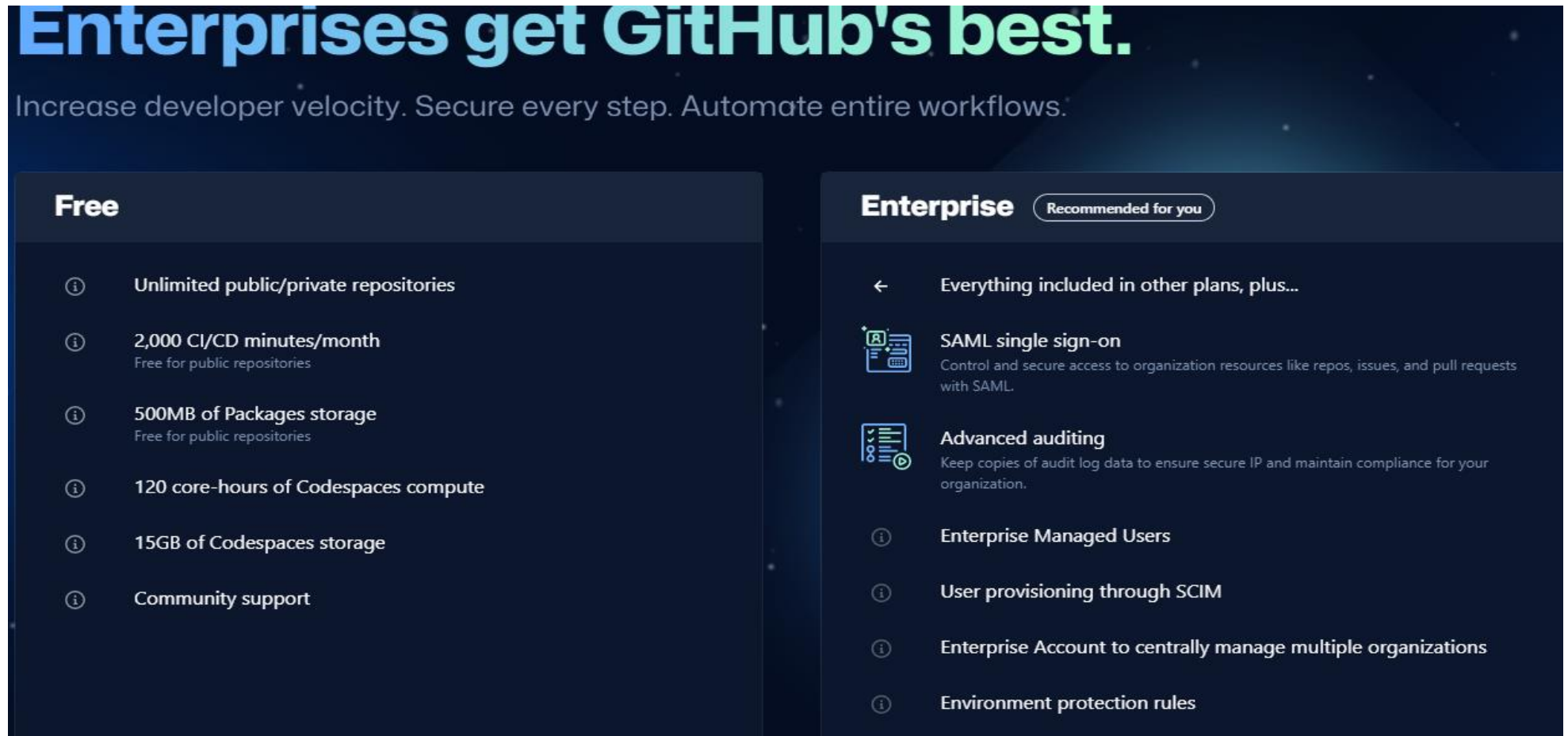
- ❖ Créer un compte GitHub, on se rend sur [la page d'accueil](#) et on clique sur [Sign up](#).



Créer compte GitHub (2)



Par défaut, GitHub est gratuit. Mais, il existe des offres payantes si on décide de passer à un niveau supérieur.



Enterprises get GitHub's best.
Increase developer velocity. Secure every step. Automate entire workflows.

Free	Enterprise Recommended for you
- Unlimited public/private repositories 2,000 CI/CD minutes/month Free for public repositories 500MB of Packages storage Free for public repositories 120 core-hours of Codespaces compute 15GB of Codespaces storage - Community support	← Everything included in other plans, plus... SAML single sign-on Control and secure access to organization resources like repos, issues, and pull requests with SAML. Advanced auditing Keep copies of audit log data to ensure secure IP and maintain compliance for your organization. - Enterprise Managed Users - User provisioning through SCIM - Enterprise Account to centrally manage multiple organizations - Environment protection rules

Créer compte GitHub (3)



- ❖ L'objectif est d'établir un **lien** entre un dépôt **local** et un dépôt **distant**.
- ❖ En fait, un dépôt Git peut être lié à un **nombre arbitraire de dépôts distants**, chacun identifié par un **nom**.
- ❖ Lors d'un clone, le dépôt cloné est automatiquement ajouté comme **dépôt distant**, sous le nom **origin**.

```
$ echo "create file from local repo to send" >> local_file.txt
$ git add .
$ git commit -m "local file to send "
$ git remote add <nom> <emplacement>
$ git push
```

De GitHub à Git et inversement



- ❖ GitHub permet de contribuer simplement à **des projets**.
- ❖ La plupart des gens qui utilisent GitHub préfère souvent de **travailler hors ligne** (en local) **plutôt que** d'être constamment **connecté** à GitHub et de passer par cette plateforme pour effectuer toutes les opérations.
- ❖ Pour travailler localement, il suffit de **cloner** le projet. ➔ Les différentes modifications apportées sur le projet seront effectuées sur la machine.
- ❖ Pour synchroniser les modifications faites depuis la machine avec le dépôt distant (GitHub), il suffit de faire un **git commit** depuis le dépôt local et d'envoyer les modifications sur le dépôt GitHub à l'aide de la commande **git push**.
- ❖ Taper **git push origin master** par exemple revient à **envoyer les modifications situées dans ma branche master vers origin**.
- ❖ Pour récupérer en local les dernières modifications du dépôt GitHub, on va utiliser la commande **git pull**.



Créer dépôt GitHub (1)

- ❖ Pour mettre le projet sur GitHub, vous devez créer un dépôt (**create repository**) dans lequel il pourra être installé.

The screenshot shows the GitHub homepage. On the left, under 'Create your first project', the 'Create repository' button is highlighted with a red box. The main content area features a 'Following' tab and a 'Discover interesting projects and people to populate your personal news feed.' section. On the right, there is a 'Latest changes' section with a list of updates. The footer contains links for Blog, About, Shop, Contact, GitHub, Pricing, API, Training, Status, Security, Terms, Privacy, and Docs.

github.com

Search or jump to...

Pull requests Issues Codespaces Marketplace Explore

Create your first project

Ready to start building? Create a repository for a new idea or bring over an existing repository to keep contributing to it.

Create repository Import repository

Recent activity

When you take actions across GitHub, we'll provide links to that activity here.

Following For you (Beta)

Discover interesting projects and people to populate your personal news feed.

Your news feed helps you keep up with recent activity on repositories you watch or star and people you follow.

Explore GitHub

ProTip! The feed shows you events from people you follow and repositories you watch or star.

Subscribe to your news feed

Latest changes

- 3 hours ago API requests are available via audit log streaming – Public Beta
- 3 days ago Repository topics and team filters for organization-level Dependabot, secret...
- 3 days ago Secret scanning now shows metrics for custom patterns
- 3 days ago Deprecation notice for npm PGP signatures

View changelog →

© 2023 GitHub, Inc.

Blog About Shop Contact GitHub Pricing

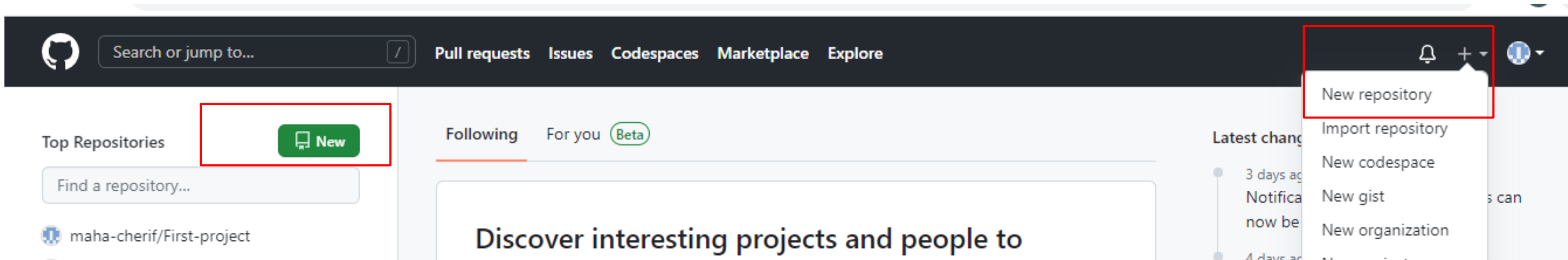
API Training Status Security

Terms Privacy Docs



Créer dépôt GitHub (2)

- ❖ Cliquez sur le "+" dans le coin supérieur droit, pour faire apparaître l'option New repository, ou tout simplement cliquez sur " New ".



Créer dépôt GitHub (3)

❖ Remplissez le formulaire et choisissez si vous souhaitez créer un dépôt public ou privé.

README est un fichier qui indique les informations clés de notre projet : description, environnement à utiliser, dépendances possibles et droits d'auteurs.

➔ C'est un peu comme le mode d'emploi de notre projet.

gitignore est un fichier qui permet d'ignorer certains fichiers de notre projet Git.

Owner * / Repository name * Premier_projet ✓

Great repository names are s Your new repository will be created as Premier_projet-. ut **animated-waddle?**

Description (optional)

☒  **Public**
Anyone on the internet can see this repository. You choose who can commit.

☐  **Private**
You choose who can see and commit to this repository.

Initialize this repository with:
Skip this step if you're importing an existing repository.

☐ **Add a README file**
This is where you can write a long description for your project. [Learn more.](#)

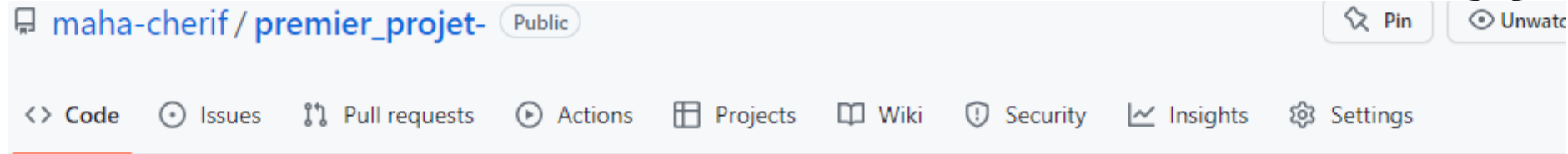
Add .gitignore
Choose which files not to track from a list of templates. [Learn more.](#)

.gitignore template: None ▼

Choose a license
A license tells others what they can and can't do with your code. [Learn more.](#)

License: None ▼

Créer dépôt GitHub (4)



Quick setup — if you've done this kind of thing before

 Set up in Desktop or [HTTPS](#) [SSH](#) https://github.com/maha-cherif/premier_projet-.git

Get started by [creating a new file](#) or [uploading an existing file](#). We recommend every repository include a [README](#), [LICENSE](#), and [.gitignore](#).

...or create a new repository on the command line

```
echo "# premier_projet-" >> README.md
git init
git add README.md
git commit -m "first commit"
git branch -M main
git remote add origin https://github.com/maha-cherif/premier_projet-.git
git push -u origin main
```

...or push an existing repository from the command line

```
git remote add origin https://github.com/maha-cherif/premier_projet-.git
git branch -M main
git push -u origin main
```

Des indications
nécessaires pour
créer un nouveau
fichier avec des
lignes de commande

ou télécharger un
fichier existant



Créer dépôt GitHub (5)

Récap :

- ❖ Pour démarrer un projet, vous devez créer votre compte GitHub.
- ❖ Vous pouvez suivre vos différents projets facilement grâce au tableau de bord.
- ❖ Pour mettre votre projet sur GitHub, vous devez créer un repository.

Créer dépôt GitHub (6)



Git :

Commençons par préparer notre dépôt local, par exemple « projet »

```
user@DESKTOP-RRVCC4I MINGW64 ~  
$ cd /c/Users/user/projet  
  
user@DESKTOP-RRVCC4I MINGW64 ~/projet  
$ git init  
Initialized empty Git repository in C:/Users/user/projet/.git/  
  
user@DESKTOP-RRVCC4I MINGW64 ~/projet (master)  
$ echo "Bonjour" >> file1.txt  
  
user@DESKTOP-RRVCC4I MINGW64 ~/projet (master)  
$ echo "Salut" >> file2.txt
```



Créer dépôt GitHub (7)

Indexer (git add) et commiter les fichiers du « projet » (git commit -m "mon premier commit")

```
user@DESKTOP-RRVCC4I MINGW64 ~/projet (master)
$ git add .
warning: in the working copy of 'file1.txt', LF will be replaced by CRLF the next time Git touches it
warning: in the working copy of 'file2.txt', LF will be replaced by CRLF the next time Git touches it

user@DESKTOP-RRVCC4I MINGW64 ~/projet (master)
$ git status
On branch master

No commits yet

Changes to be committed:
  (use "git rm --cached <file>..." to unstage)
        new file:   file1.txt
        new file:   file2.txt

user@DESKTOP-RRVCC4I MINGW64 ~/projet (master)
$ git commit -m "premier commit"
[master (root-commit) dac7232] premier commit
 2 files changed, 2 insertions(+)
 create mode 100644 file1.txt
 create mode 100644 file2.txt
```



Créer dépôt GitHub (8)

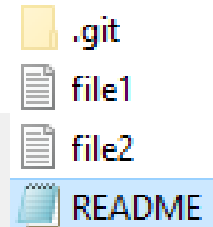
Commençons par lancer les commandes et vérifier son ajout sur le dossier

```
user@DESKTOP-RRVCC4I MINGW64 ~/projet (master)
$ echo "#premier_project-" >> README.md

user@DESKTOP-RRVCC4I MINGW64 ~/projet (master)
$ git init
Reinitialized existing Git repository in C:/Users/user/projet/.git/

user@DESKTOP-RRVCC4I MINGW64 ~/projet (master)
$ git add README.md
warning: in the working copy of 'README.md', LF will be replaced by CRLF the next time Git touches it

user@DESKTOP-RRVCC4I MINGW64 ~/projet (master)
$ git commit -m "deuxième commit"
[master 7e57c2c] deuxième commit
1 file changed, 1 insertion(+)
create mode 100644 README.md
```





Créer dépôt GitHub (9)

Envoyer les fichiers vers le dépôt GitHub (https)

```
user@DESKTOP-RRVCC4I MINGW64 ~/projet (master)
$ git branch -M main

user@DESKTOP-RRVCC4I MINGW64 ~/projet (main)
$ git remote add origin https://github.com/maha-cherif/premier_projet-.git

user@DESKTOP-RRVCC4I MINGW64 ~/projet (main)
$ git push -u origin main
Enumerating objects: 7, done.
Counting objects: 100% (7/7), done.
Delta compression using up to 4 threads
Compressing objects: 100% (4/4), done.
Writing objects: 100% (7/7), 558 bytes | 186.00 KiB/s, done.
Total 7 (delta 0), reused 0 (delta 0), pack-reused 0
To https://github.com/maha-cherif/premier_projet-.git
 * [new branch]      main -> main
branch 'main' set up to track 'origin/main'.
```



Créer dépôt GitHub (10)

Vérifier la réception des fichiers dans le dépôt GitHub

 maha-cherif deuxième commit 7e57c2c 10 minutes ago 🕒 2 commits

 README.md	deuxième commit	10 minutes ago
 file1.txt	premier commit	2 hours ago
 file2.txt	premier commit	2 hours ago

README.md



#premier_project-

Authentification auprès de GitHub



Pour maintenir votre compte **sécurisé**, vous devez vous **authentifier** avant de pouvoir accéder à certaines des ressources sur GitHub.

Quand nous nous authentifions auprès de GitHub, nous fournissons ou confirmons des **informations d'identification** qui nous sont propres pour prouver que nous sommes exactement qui nous déclarons être.

Nous pouvons accéder à nos ressources dans GitHub de différentes façons : dans le navigateur, avec GitHub Desktop ou une autre application de bureau, avec l'API ou par le biais de la ligne de commande.

Chaque façon d'accéder à GitHub prend en charge différents modes d'authentification.

- Nom d'utilisateur et mot de passe avec authentification à 2 facteurs
- **Personal access token**
- **Clé SSH**



Création d'un jeton d'accès personnel (1)

Utilisez personal access token à la place d'un mot de passe lors de l'authentification à GitHub avec la ligne de commande.

Les Personal access token représentent une alternative à l'utilisation de mots de passe pour l'authentification auprès de GitHub durant l'utilisation de la ligne de commande.

Les Personal access token permettent l'accès aux ressources GitHub en utilisant notre nom.

Types de personal access token

GitHub prend en charge deux types de personal access token :

- les fine-grained personal access token
- les personal access tokens (classic).

Création d'un jeton d'accès personnel (2)



Fine-grained personal access token

Les **Fine-grained** personal access token présentent plusieurs avantages au niveau de la sécurité par rapport **classic** :

- Chaque jeton ne peut accéder qu'aux ressources appartenant à un seul utilisateur.
- Chaque jeton peut uniquement accéder à des dépôts spécifiques.
- Chaque jeton se voit attribuer des autorisations spécifiques, qui offrent plus de contrôle que les étendues attribuées aux personal access tokens (classic).
- Chaque jeton doit avoir une date d'expiration.
- Les propriétaires d'organisation peuvent imposer une approbation à tous les fine-grained personal access token ayant accès aux ressources de l'organisation.

Création d'un jeton d'accès personnel (3)



Personal access token classic

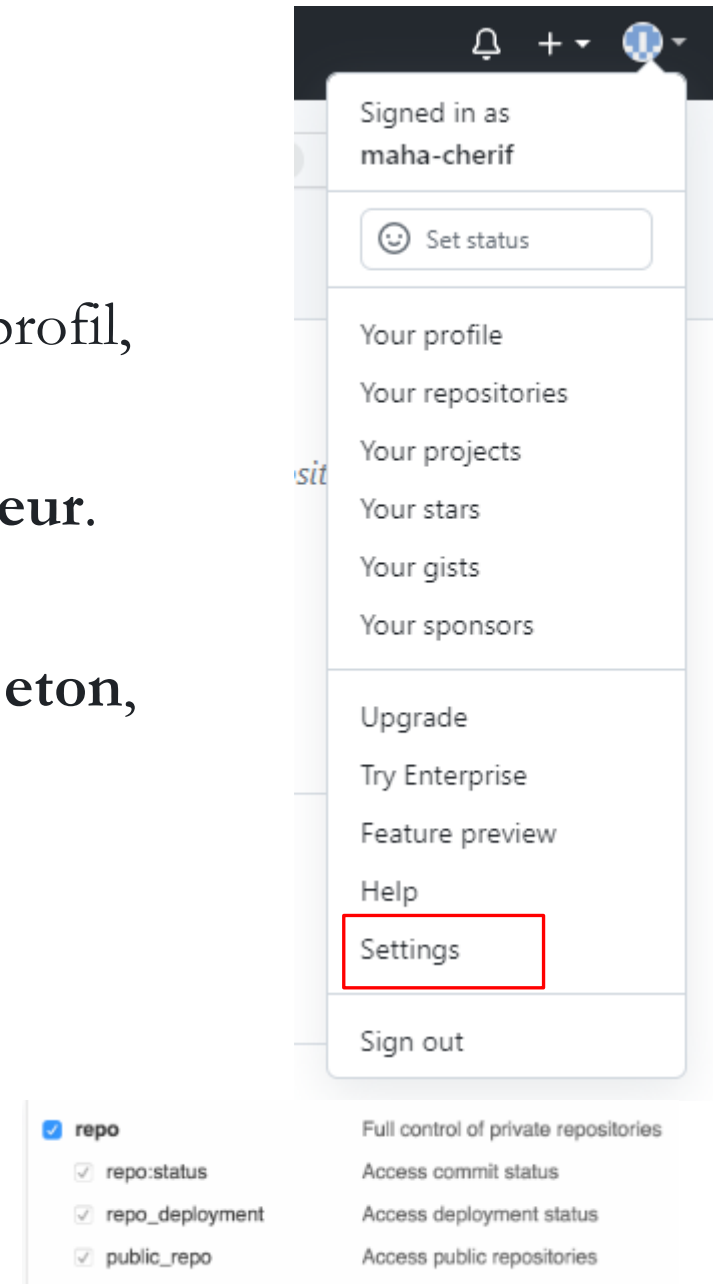
Les Personal access tokens (classic) sont **moins sécurisés**. Cependant, certaines fonctionnalités sont utilisables uniquement avec les personal access tokens (classic) :

- Seuls les personal access tokens (classic) disposent d'un accès en écriture pour les dépôts publics qui ne nous appartiennent pas, ou qui appartiennent à une organisation dont nous ne sommes pas membre.
 - Les collaborateurs externes peuvent utiliser uniquement les personal access tokens (classic) pour accéder aux dépôts d'organisation dont ils sont collaborateurs.
- ➔ Si nous choisissons d'utiliser un personal access token (classic), on ne doit pas qu'il accorde l'accès à tous les dépôts au sein des organisations auxquelles nous avons accès, ainsi qu'à tous les dépôts personnels dans notre compte personnel.

Création d'un jeton d'accès personnel (4)

Création d'un personal access token (classic)

1. Dans le coin supérieur droit d'une page, cliquez sur votre photo de profil, puis sur **Paramètres**.
2. Dans la barre latérale gauche, cliquez sur **Paramètres de développeur**.
3. Dans la barre latérale gauche, sous **Personal access token**, cliquez sur **Jetons (classique)**. 1. Sélectionnez **Générer un nouveau jeton**, puis cliquez sur **Générer un nouveau jeton (classique)**.
4. Donnez à votre jeton un nom descriptif.
5. Attribuer à votre jeton un délai d'expiration.
6. Sélectionnez les étendues à attribuer à ce jeton. Pour utiliser votre jeton afin d'accéder aux **dépôts** à partir de la ligne de commande, sélectionnez dépôt.





Authentification et Connexion avec SSH (1)

Cependant, si vous avez configuré Git pour utiliser **une clé SSH** pour l'authentification, il est possible que Git n'ait pas besoin de vous demander de vous authentifier pour utiliser la commande `git pull`.

Dans ce cas, Git utilisera la clé SSH pour **s'authentifier automatiquement** lors de la récupération des modifications à partir du dépôt distant



Authentification et Connexion avec SSH (2)

- ❖ L'authentification par échange de clés SSH fonctionne en plaçant une **clé publique** sur l'ordinateur distant et en utilisant une **clé privée** depuis son ordinateur.
- ❖ Ces deux clés (publique et privée) sont liées l'une à l'autre.
- ❖ C'est seulement en présentant la clé privée à la clé publique qu'il est possible se connecter.
- ❖ Chaque clé se présente sous la forme d'une longue chaîne de caractères enregistrée dans un fichier.
- ❖ Pour plus de sécurité, vous pouvez également protéger la clé privée avec une phrase secrète.
Autrement dit, pour pouvoir utiliser la clé privée, il faudra saisir un mot de passe, ce qui renforce encore davantage la sécurité.
- ❖ Les clés SSH peuvent être créées avec différents algorithmes de chiffrement :
Dans notre cas, nous allons utiliser :
Ed25519 : l'algorithme à privilégier aujourd'hui.



Génération des clés SSH (1)

- ❖ Générez une paire de clés SSH avec la commande **ssh-keygen** en spécifiant l'algorithme de chiffrement désiré. Avec Ed25519 :

```
$ ssh-keygen -t ed25519
```

- ❖ Laissez l'emplacement par défaut en appuyant sur Entrée.

```
Generating public/private ed25519 key pair. Enter file in  
which to save the key (C:\Users\Pierre\.ssh\id_ed25519):
```

- ❖ Entrez une phrase secrète pour protéger votre clé privée :

```
Enter passphrase (empty for no passphrase): Enter same  
passphrase again:
```



Génération des clés SSH (2)

❖ Voilà, votre paire de clés SSH a bien été générée !

Your identification has been saved in C:\Users\Pierre\.ssh\id_ed25519. Your public key has been saved in C:\Users\Pierre\.ssh\id_ed25519.pub. The key fingerprint is:

SHA256:4Yjb63lZzyRw+ADKaZ6nwZDA7jBrtorVR4mkgXRGWN0 pierre@DESKTOP-VJAT016 The key's randomart image is:

+--[ED25519 256]--+

```
|..+=. . |
|o+o ..E |
|o..o.o ... |
|o.o+=o ++.. |
|oo.=o.+ S= |
|. + . = + . + . |
|o o o+o o = |
|.o .. oo o |
|+ .+. |
```

+-----[SHA256]-----



Génération des clés SSH (3)

❖ Vos clés SSH se trouvent dans le répertoire `~/.ssh/`.

```
$ cd <lien de ssh>
$ ls
id_ed25519      id_ed25519.pub

$ cat id_ed25519
$ cat id_ed25519.pub
```

- ➔ La paire de clés SSH (**publique et privée**) a bien été créée.
- ➔ Prochaine étape : copier la clé publique sur le serveur distant.



Copier la clé publique SSH sur le serveur distant (1)

- ❖ Pour copier une clé **publique** sur le compte d'un utilisateur distant, suivez ces instructions :
- ➔ Open the github settings> ssh and gpg keys
- ❖ Sélectionnez un nom pour votre clé **publique** et collez la clé déjà copiée.

The screenshot shows the GitHub 'SSH keys / Add new' page. On the left is a sidebar with the user's profile 'maha-cherif' and a list of settings: Public profile, Account, Appearance, Accessibility, Notifications, Access, Billing and plans, Emails, Password and authentication, Sessions, SSH and GPG keys (highlighted), Organizations, and Moderation. The main content area has a title 'SSH keys / Add new' and three input fields: 'Title' (containing 'clé ssh'), 'Key type' (a dropdown menu set to 'Authentication Key'), and 'Key' (a large text area containing the SSH public key: 'ssh-ed25519 AAAAC3NzaC1lZDI1NTE5AAAAICxw9z6cMzR5+KnODPprl12rc5Z/6uk4tgAaFlc9RjEs user@DESKTOP-RRVCC4I'). At the bottom right of the main area is a green button labeled 'Add SSH key'.



Copier la clé publique SSH sur le serveur distant (2)

❖ On tape la commande :

```
$ ssh -T git@github.com
```

❖ Donc, on peut envoyer et cloner nos fichiers vers ou depuis le dépôt distant en toute sécurité.

(création d'un dossier .git, cd ./ dossier, touch, git add, git commit -m ...)

```
$ git remote add origin git@github.com:maha-cherif/work.git  
$ git push origin master  
$ git clone git@github.com:maha-cherif/work.git
```